



Helm Charts for Simplified AI Deployments

Day 34 - DevOps for AI/ML Workloads

Why Helm for AI?

Raw Kubernetes YAML manifests are:

- Long and repetitive
- Difficult to manage at scale
- Prone to configuration errors

AI applications typically require **multiple manifests** for Deployments, Services, PVCs, and Ingress resources.

Helm solves these problems as a Kubernetes **package manager**:

- Simplifies installation & updates
- Works like **apt/pip for Kubernetes**
- Packages related resources together
- Enables templating and reuse

What Is Helm?



Chart Packaging

Packages Kubernetes YAML manifests into reusable **Charts** that can be versioned and shared across teams and environments



Templating

Supports powerful templating for configuration flexibility and maintainable infrastructure definitions

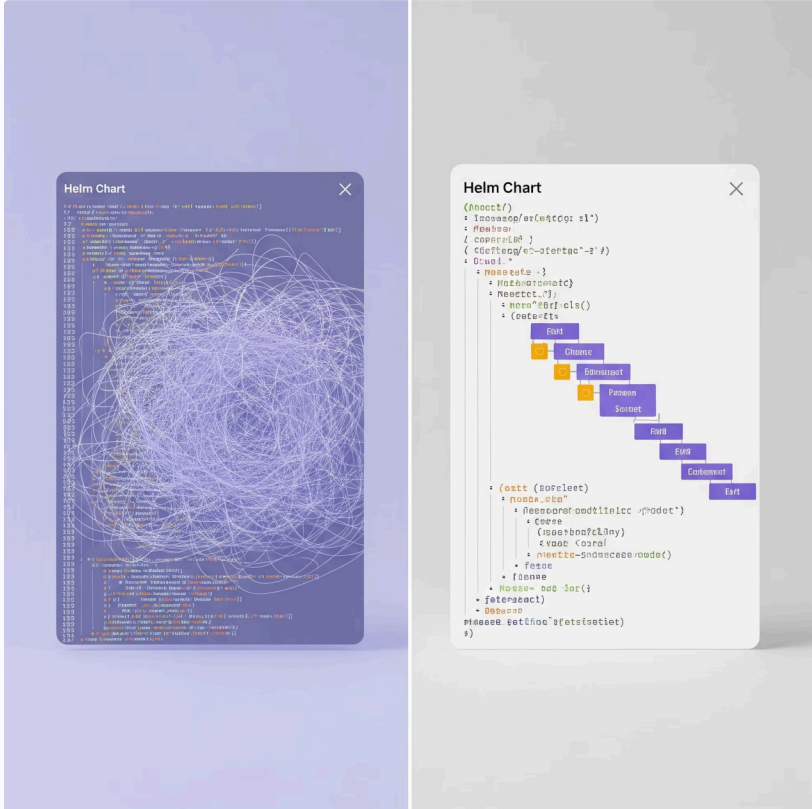


Versioning & Rollbacks

Provides built-in versioning and rollback capabilities for reliable deployments of ML models and infrastructure

Helm has become the **standard tool** for managing production-grade Kubernetes AI systems across the industry

Helm vs Raw YAML



Raw YAML Challenges

- Verbose and repetitive
- Hard to maintain at scale
- Environment-specific values embedded
- No built-in versioning

Helm Advantages

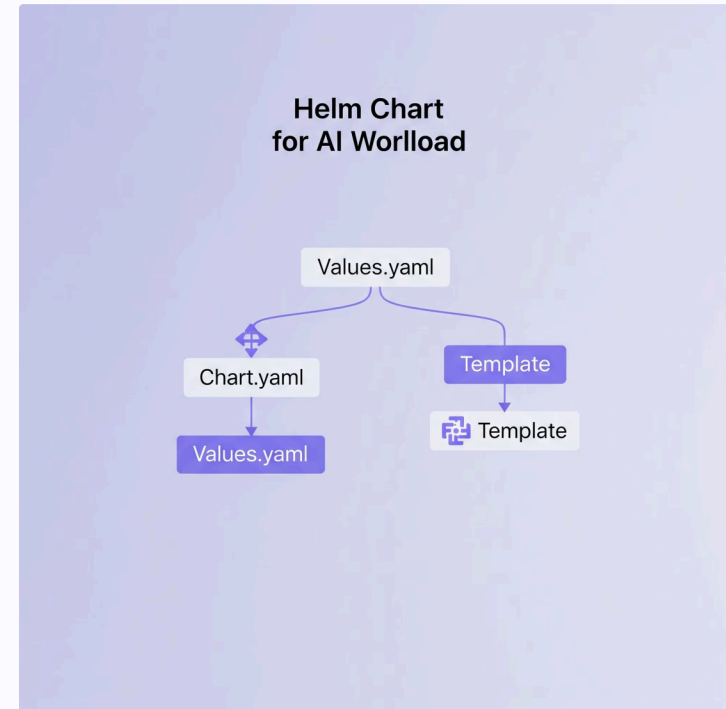
- Parameterized, DRY approach
- Reuse configs across environments
- Built-in rollback for failed releases
- Makes AI infrastructure modular

Anatomy of a Helm Chart

```
my-ai-chart/  
├── Chart.yaml  # metadata  
├── values.yaml # default configs  
└── templates/  # K8s YAML templates  
    ├── deployment.yaml  
    ├── service.yaml  
    └── ingress.yaml
```

Core components:

- **Chart.yaml**: Contains metadata like name, version, and dependencies
- **values.yaml**: Default configuration values that can be overridden
- **templates/**: Directory containing templated Kubernetes manifests



Helm charts follow a standardized structure that makes them portable and reusable across different AI/ML deployment scenarios.

Example values.yaml

```
replicaCount: 3
image:
  repository: ai-app
  tag: latest
service:
  type: LoadBalancer
  port: 80
resources:
  limits:
    cpu: "2"
    memory: "4Gi"
```

Benefits for ML Engineers

The values.yaml file provides a [single source of configuration](#) that:

- Controls deployment scale, image, and service type
- Allows engineers to tune parameters without editing YAML
- Separates configuration from implementation
- Makes GPU/memory allocation explicit and standardized
- Simplifies environment-specific configurations

 ML engineers can modify only the values they need without understanding the underlying Kubernetes YAML structure.

Installing a Chart

Install a custom AI chart

```
helm install my-ai-app ./my-ai-chart
```

Deploy your own AI application with one command

Add repository for public charts

```
helm repo add bitnami  
https://charts.bitnami.com/bitnami
```

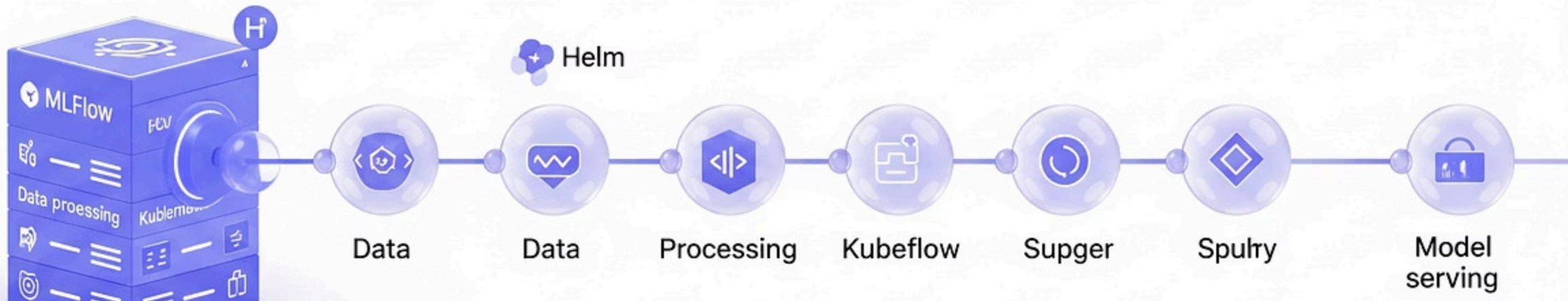
Access libraries of pre-built charts for common ML tools

Install from repository

```
helm install mlflow  
bitnami/mlflow
```

Deploy complex ML infrastructure with a single command

Helm dramatically simplifies deploying both custom and community-maintained AI applications on Kubernetes.



Helm for AI Pipelines

Common AI Tools Available as Charts

- **MLFlow** for experiment tracking
- **Kubeflow** components for ML pipelines
- Model serving platforms (TensorFlow Serving, Seldon)
- Monitoring tools (Prometheus, Grafana)

Benefits for ML Teams

- Package inference APIs with Helm charts
- Reuse charts across teams & environments
- Accelerate infrastructure deployments
- Standardize ML platform components

Rollbacks & Upgrades



Upgrade

```
helm upgrade my-ai-app ./chart
```

Safely deploy new model versions or configuration changes



Rollback

```
helm rollback my-ai-app 1
```

Instantly revert to previous version if issues occur



Canary Deploy

Support for canary deployments when testing new ML models in production

Helm's upgrade and rollback capabilities are [crucial for AI workloads](#) where model performance needs to be carefully monitored and quickly reverted if performance degrades.

Best Practices



Version Control

Keep charts **version-controlled** in Git using GitOps principles

Environment Values

Use **separate values.yaml** files for dev, staging, and production

Chart Repository

Store enterprise AI charts in private repositories for security and governance

CI/CD Integration

Combine with CI/CD for automated deployments of ML models

Key Takeaways



Kubernetes Package Manager

Helm serves as a powerful package manager specifically designed for Kubernetes AI applications



Templating Benefits

Templating significantly reduces YAML duplication and maintenance overhead



Simplified Infrastructure

Charts simplify deploying complex ML infrastructure like MLFlow, APIs, and databases



Deployment Management

Built-in support for upgrades, rollbacks, and CI/CD integration makes Helm a **must-have tool** for scalable AI deployments